Лабораторна робота №7

Тема: Побудова систем нечіткого виведення (Fuzzy Logic) засобами Python

Мета роботи:

1. Ознайомитися з принципами нечіткої логіки та алгоритмом виведення Мамдані.
2. Навчитися використовувати бібліотеку scikit-fuzzy для побудови нечітких контролерів.
3. Опанувати процес обробки реальних даних (Data Mining) через інтеграцію нечіткої системи з даними з Google Forms.

Інструменти: Python 3.x, середовище розробки (Jupyter Notebook / PyCharm / VS Code).

Необхідні бібліотеки: numpy, scikit-fuzzy, matplotlib, pandas.

□ Частина 1. Теоретичні відомості та підготовка

Система нечіткого виведення (Fuzzy Inference System – FIS) дозволяє моделювати процес прийняття рішень людиною, оперуючи нечіткими поняттями ("висока температура", "середній ризик", "приваблива пропозиція") замість точних чисел.

Основні етапи нечіткого виведення:

1. **Фазифікація (Fuzzification):** Перетворення вхідних чітких значень (наприклад, швидкість = 55 км/год) у ступені належності до лінгвістичних термів (наприклад, "Швидко" = 0.6).
2. **База правил (Rule Base):** Набір правил "ЯКЩО – ТО" (IF – THEN).
3. **Агрегація та Акумуляція:** Визначення результуючої нечіткої фігури на основі активних правил.
4. **Дефазифікація (Defuzzification):** Перетворення результуючої фігури у чітке число (зазвичай методом Центроїда).

Встановлення бібліотек

Відкрийте термінал і виконайте команду:

Bash
```
pip install scikit-fuzzy numpy matplotlib pandas
```

□ Частина 2. Базовий приклад (Tutorial)

Завдання: Реалізувати класичну задачу розрахунку чайових.

- **Вхід 1:** Якість їжі (0..10).
- **Вхід 2:** Якість обслуговування (0..10).
- **Вихід:** Розмір чайових у відсотках (0..25%).

Проаналізуйте та запустіть наступний код:

Python

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# 1. Створення лінгвістичних змінних
# Antecedent - вхідні дані, Consequent - вихідні дані
food = ctrl.Antecedent(np.arange(0, 11, 1), 'food')
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')

# 2. Функції належності
# Автоматичні (3 рівні: poor, average, good)
food.automf(3)
service.automf(3)

# Ручні (трикутні функції для виходу)
tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13])
tip['medium'] = fuzz.trimf(tip.universe, [0, 13, 25])
tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25])

# 3. База правил
rule1 = ctrl.Rule(food['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | food['good'], tip['high'])

# 4. Створення контролера
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping = ctrl.ControlSystemSimulation(tipping_ctrl)

# 5. Тестування
tipping.input['food'] = 6.5
tipping.input['service'] = 9.8

# Розрахунок
tipping.compute()

print(f"Рекомендовані чайові: {tipping.output['tip']:.2f}%")
tip.view(sim=tipping)
plt.show()
```

□ Частина 3. Індивідуальне завдання (Engineering)

Оберіть одну із запропонованих предметних областей (або запропонуйте власну) та побудуйте для неї нечітку експертну систему.

Вимоги до системи:

- Мінімум **2 вхідні змінні**.
- Мінімум **1 вихідна змінна**.
- Мінімум **4 правила** в базі знань.
- Використання різних типів функцій належності (не тільки automf, спробуйте задати trapmf або gaussmfвручну).

Варіанти задач:

1. **Система гальмування:** Входи – дистанція до перешкоди, швидкість. Вихід – сила тиску на гальма.
2. **Розумний кондиціонер:** Входи – температура, вологість. Вихід – потужність обдуву.
3. **Оцінка студентської роботи:** Входи – оригінальність тексту, кількість сторінок. Вихід – попередня оцінка.
4. **Вибір смартфона:** Входи – ціна, якість камери. Вихід – ймовірність покупки.
5. **Варіант на свій вибір…**

У звіті навести код, графіки функцій належності та приклад роботи системи.

□ Частина 4. Фінальний проєкт: "Аналітика сумісності" (Data Mining)

Це завдання імітує реальну роботу Data Scientist: збір даних, побудова моделі та пакетна обробка результатів.

Легенда: Ви розробляєте алгоритм для дейтинг-додатку. Вам потрібно створити систему, яка оцінює "сумісність" кандидата з вами на основі ваших особистих критеріїв.

Етап 1. Створення Google Форми

1. Придумайте **2-3 критерії**, які важливі для вас у партнері (наприклад: "Почуття гумору", "Любов до ігор", "Вміння готувати", "Рівень спокою").
2. Створіть **Google Form**, де кожне питання відповідає вашому критерію.
    - *Тип питання:* Лінійна шкала (від 1 до 10).
    - *Обов'язкове поле:* Ім'я (або нікнейм).

3. Надішліть форму у чат групи. Попросіть ваших **одногрупників** заповнити її.
4. В Google Формі перейдіть у вкладку "Відповіді" -> "Зв'язати з Google Таблицями".
5. Завантажте отриману таблицю у форматі **.CSV** (Файл -> Завантажити -> Значення, розділені комою).

Етап 2. Розробка Fuzzy-системи

Напишіть Python-скрипт, який:

1. Містить вашу особисту базу правил (наприклад: IF (Гумор IS Poor) & …THEN (Chance IS Zero)).
2. Зчитує файл .csv за допомогою бібліотеки pandas.
3. Проганяє дані *кожного* одногрупника через вашу нечітку систему.
4. Додає нову колонку "Compatibility_Score" у таблицю.
5. Виводить ТОП-3 кандидатів.

Вимоги до звіту

Звіт повинен містити:

1. Титульна сторінка.
2. Код та результати виконання **Частини 2** (Чайові).
3. Опис та графіки функцій належності для **Частини 3** (Індивідуальна задача).
4. Звіт по **Частині 4** (Проєкт "Сумісність"):
    - Скріншот вашої Google Форми.
    - Скріншот фрагмента CSV файлу.
    - Код обробки.
    - Фінальна таблиця з ранжуванням одногрупників.
    - Короткий висновок: чи збіглися результати роботи системи з вашим інтуїтивним баченням?